

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	Miruthula JMM	
Reg. No.	24011101039	
Class	AI DS - A (Batch 2)	

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

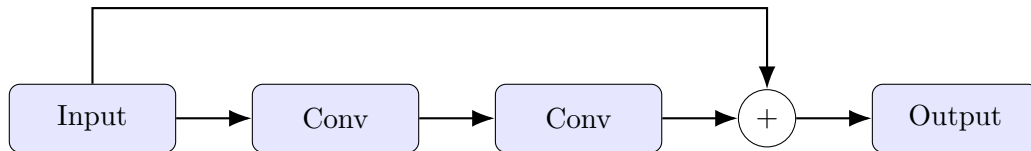


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

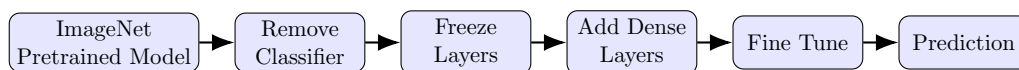


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$

- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

The notebook used a stratified 90%/10% split of the 50,000 training images, giving 45,000 training images and 5,000 validation images, while the original 10,000-image test set was retained for final evaluation. The images were initially normalized to the range $[0, 1]$. For the ResNet50 input pipeline, the images were then resized to 96×96 and passed through the ResNet50-specific preprocessing function.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0, 1]$.
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

The notebook loaded CIFAR-10 using `tf.keras.datasets.cifar10` after the Kaggle loading route was not populated in the runtime. The resulting training and testing dimensions were $(50000, 32, 32, 3)$ and $(10000, 32, 32, 3)$ respectively. A stratified split produced $(45000, 32, 32, 3)$ training images and $(5000, 32, 32, 3)$ validation images.

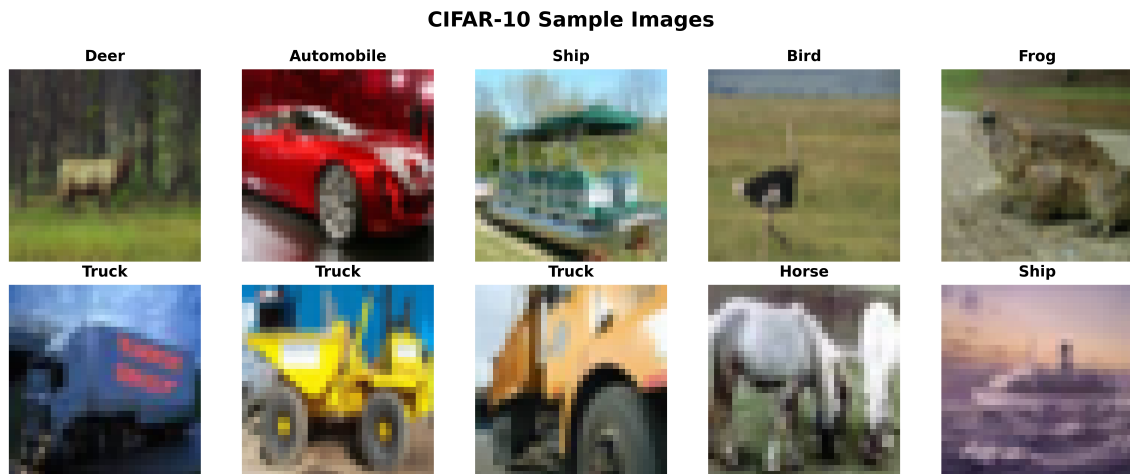


Figure 3: Ten sample CIFAR-10 images with their class labels.

Inference: The samples demonstrate the ten CIFAR-10 categories and show that the dataset contains visually different objects as well as fine-grained classes such as cat, dog, deer and bird. These classes can be more difficult to distinguish because the images are only 32×32 pixels.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

For the notebook implementation, **ResNet50** was selected. The ImageNet-pretrained ResNet50 convolutional base was loaded with `include_top=False` and frozen during the feature-extraction phase. The CIFAR-10 images were resized to 96×96 . The custom classification head was

GlobalAveragePooling2D $\rightarrow$ Dense(256, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(10, Softmax).

The model contained 24,114,826 total parameters. Initially, 23,587,712 ResNet50 parameters were non-trainable and 527,114 parameters in the new classification head were trainable.

Inference: Freezing the pretrained base allows the new classifier to learn CIFAR-10-specific decision boundaries without updating the large ImageNet feature extractor. This reduces the number of parameters that must be optimized during the first training phase.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

The notebook used Adam with an initial learning rate of 0.001 and trained the frozen-base classifier for 20 epochs. The implementation used a tuned batch size of 64 rather than the manual value of 32. The loss was categorical cross entropy with label smoothing of 0.1. A `ReduceLROnPlateau` scheduler and early stopping were used, and the best validation model was checkpointed.

The feature-extraction phase achieved 95.77% training accuracy and 88.86% validation accuracy at epoch 20. The head-training time was 683.41 seconds.

Inference: The frozen pretrained representation allowed the classifier to reach high accuracy quickly. The difference between the training and validation accuracy also shows that the classification head was fitting the training data more strongly than the validation data.

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

The notebook unfroze the final ResNet50 residual block, beginning with the layers whose names start with `conv5_block`. The fine-tuning learning rate was reduced to 10^{-5} and training continued for ten epochs, from epoch 21 to epoch 30.

The final fine-tuned training accuracy was 99.00% and the final validation accuracy was 91.38%. Compared with the frozen-base phase, validation accuracy improved from 88.86% to 91.38%, an improvement of 2.52 percentage points. Fine tuning took 400.95 seconds, giving a total training time of 1084.36 seconds.

Inference: Fine tuning improved the validation accuracy because the final convolutional block was allowed to adapt ImageNet features to CIFAR-10. The training accuracy also increased substantially, while the validation curve continued to improve through the final epoch.

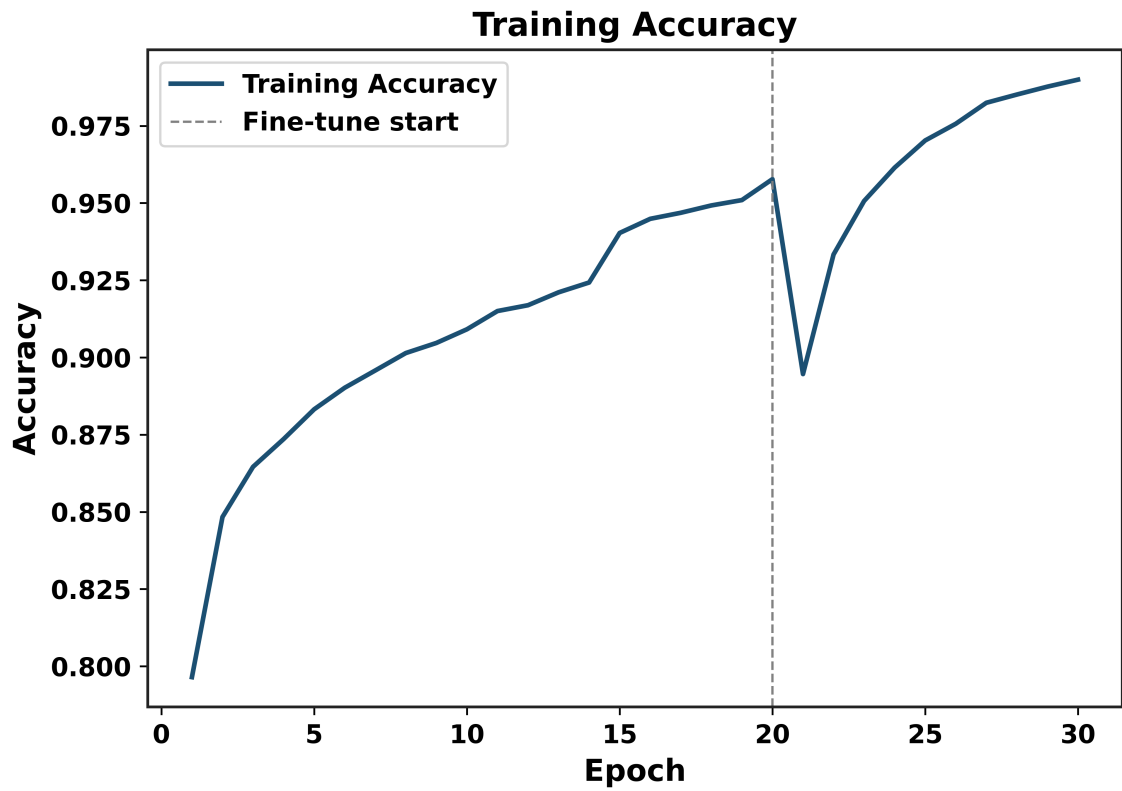


Figure 4: Training accuracy over 30 epochs; the dashed line marks the start of fine tuning.

Inference: Training accuracy rises from about 0.80 in the first epoch to 0.96 by epoch 20. At the beginning of fine tuning there is a temporary drop, after which accuracy rises to approximately 0.99 by epoch 30. This indicates that the unfrozen residual block was able to adapt the pretrained representation to the target dataset.

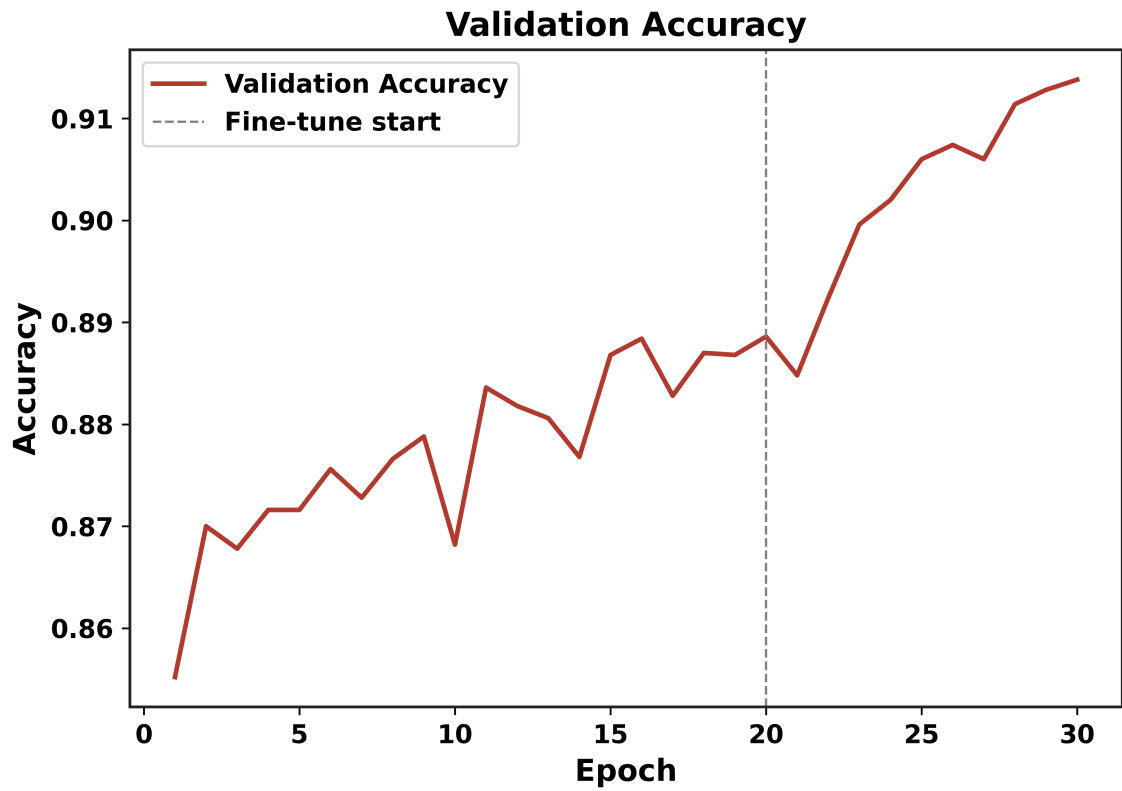


Figure 5: Validation accuracy over 30 epochs; the dashed line marks the start of fine tuning.

Inference: Validation accuracy fluctuates during the frozen-base phase and reaches 0.8886 at epoch 20. After fine tuning begins, it increases steadily and reaches 0.9138 at epoch 30, showing that fine-tuning improved generalization on the validation set.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

The final model was evaluated on all 10,000 held-out CIFAR-10 test images.

Metric	Value
Test Accuracy	90.55%
Test Loss	0.7563
Precision (weighted)	0.9055
Recall (weighted)	0.9055
F1-score (weighted)	0.9053
Total Parameters	24,114,826
Total Training Time	1084.36 s

The class-wise classification report is shown below.

Class	Precision	Recall	F1-score	Support
Airplane	0.93	0.92	0.92	1000
Automobile	0.95	0.94	0.95	1000
Bird	0.90	0.89	0.90	1000
Cat	0.85	0.79	0.82	1000
Deer	0.87	0.89	0.88	1000
Dog	0.84	0.87	0.86	1000
Frog	0.92	0.94	0.93	1000
Horse	0.93	0.92	0.92	1000
Ship	0.93	0.95	0.94	1000
Truck	0.94	0.94	0.94	1000

Inference: The model achieved 90.55% test accuracy with precision, recall and F1-score all close to 0.91. The strongest class-wise performance was obtained for Automobile, while Cat was the most difficult class among the ten based on its lower recall and F1-score.

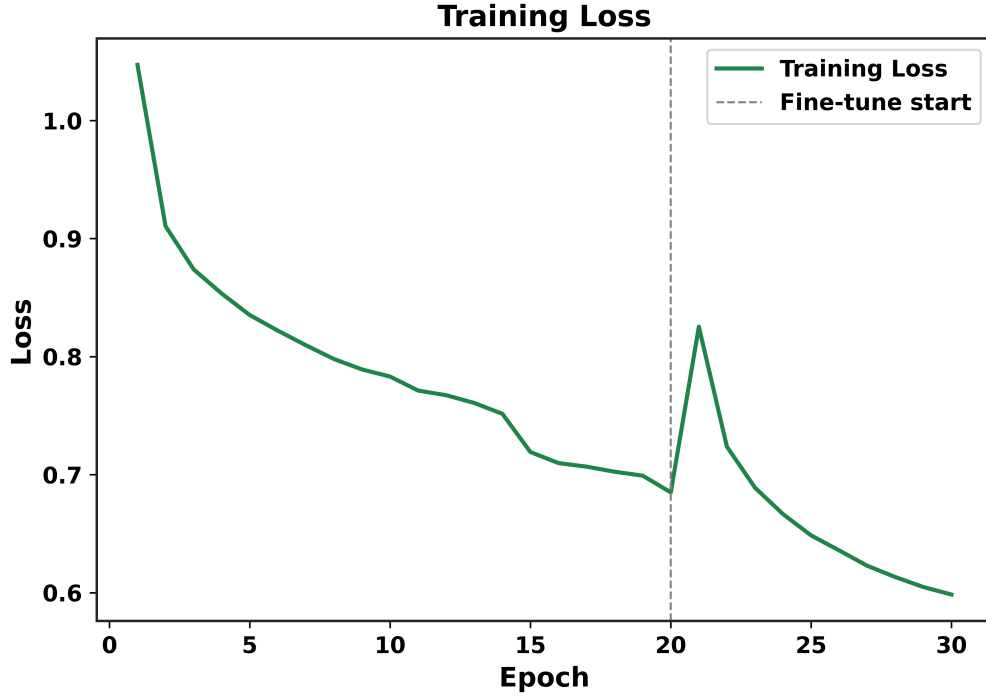


Figure 6: Training loss over 30 epochs.

Inference: Training loss decreases from approximately 1.05 to 0.60 over the experiment. The temporary increase immediately after fine-tuning starts is followed by a continuous decrease, indicating that the adapted feature extractor and classifier continue to optimize the training objective.

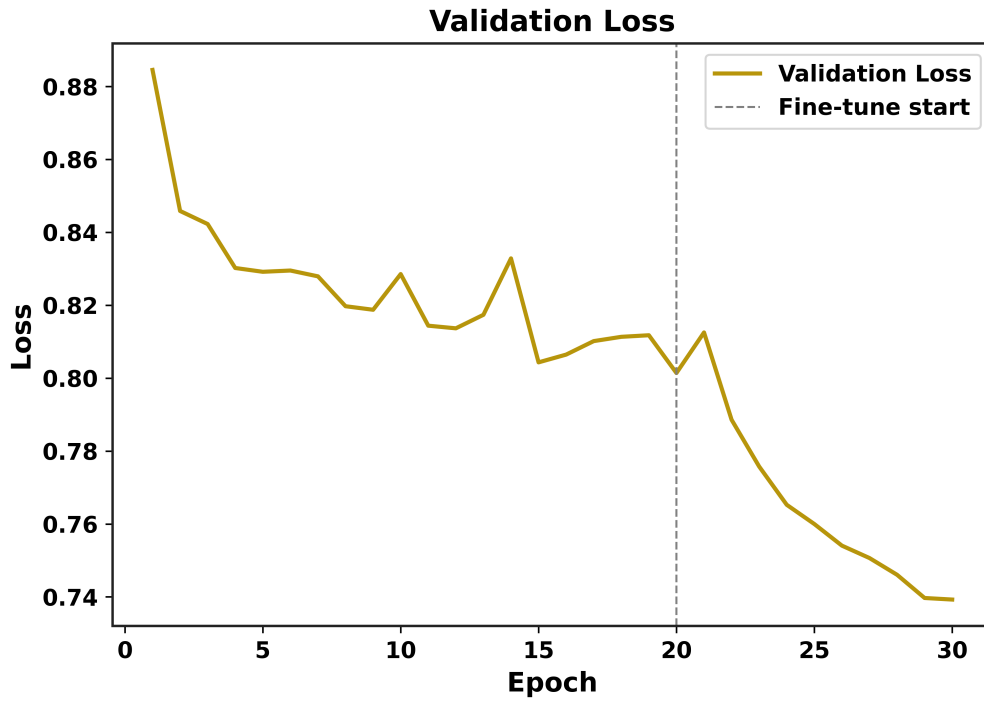


Figure 7: Validation loss over 30 epochs.

Inference: Validation loss generally decreases throughout the experiment, with a temporary increase around the transition to fine tuning. It reaches approximately 0.739 by epoch 30, which is consistent with the improvement in validation accuracy.

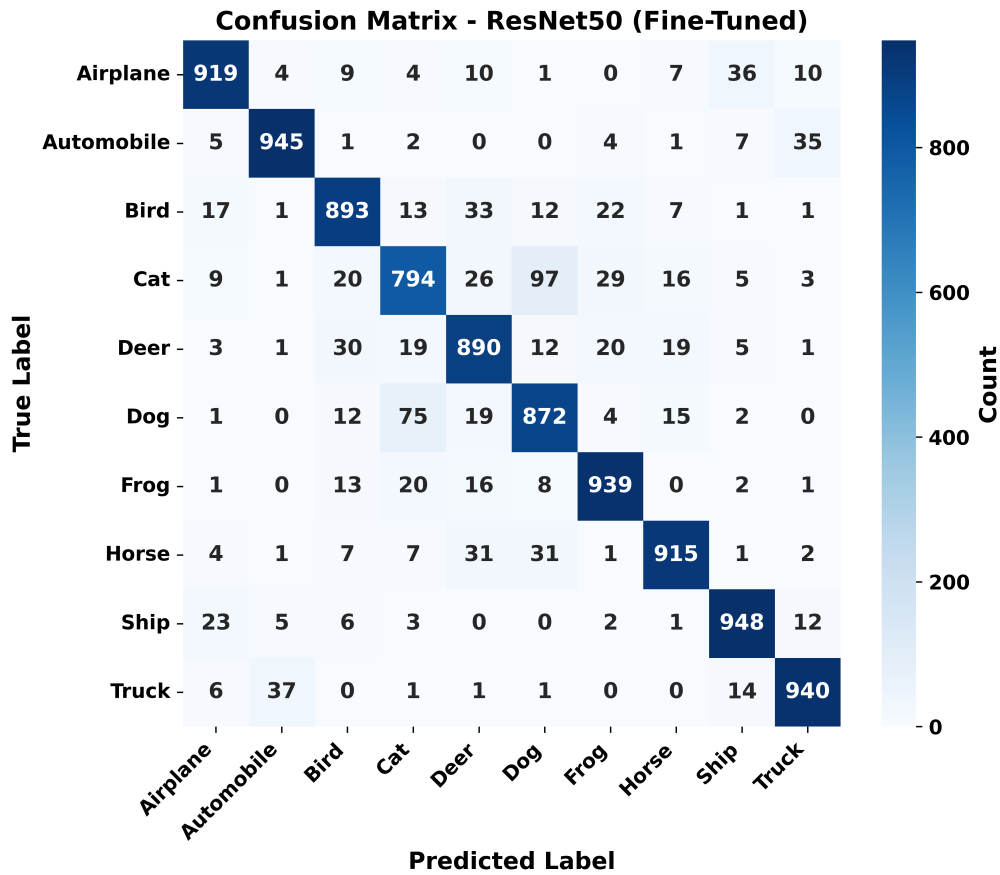


Figure 8: Confusion matrix on the CIFAR-10 test set using the fine-tuned ResNet50.

Inference: The confusion matrix has strong diagonal entries, particularly for Automobile, Frog, Ship and Truck. Noticeable errors occur between visually similar classes, especially Cat and Dog, showing that fine-grained object distinctions remain challenging even after transfer learning.

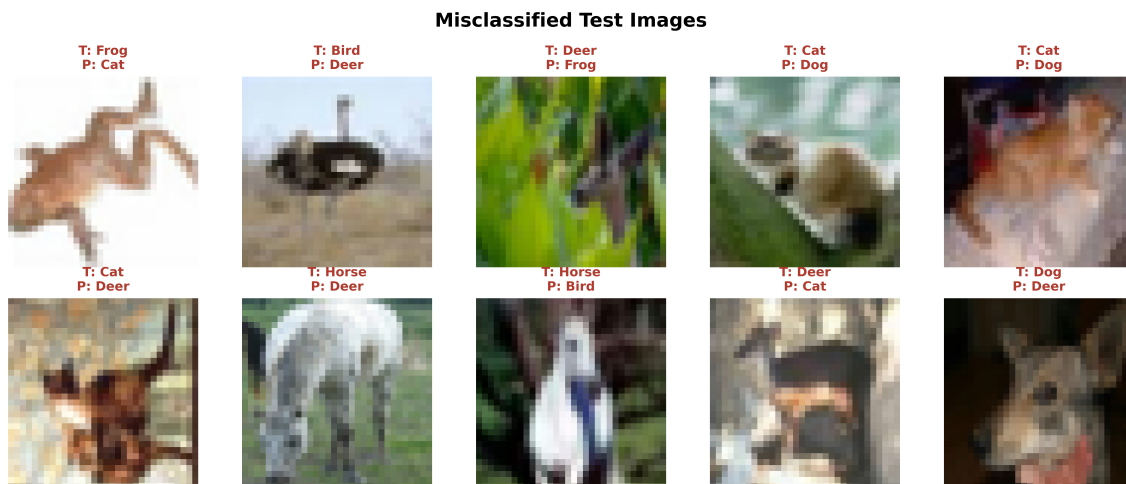


Figure 9: Examples of misclassified CIFAR-10 test images.

Inference: The misclassified examples are concentrated mainly among visually similar animal classes such as Cat, Dog, Deer, Bird and Horse. The low spatial resolution of CIFAR-10 makes fine details difficult to distinguish, explaining many of these errors.

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

The notebook did not record a complete numerical sweep for every combination listed in the manual. It did, however, record the selected configuration used for the final experiment:

Setting	Selected value
Base learning rate	0.001
Fine-tuning learning rate	10^{-5}
Batch size	64
Frozen-base epochs	20
Fine-tuning epochs	10
Optimizer	Adam
Dense units	256
Frozen layers	All initially, then partial

Inference: The recorded experiment supports the use of a larger batch size of 64 and a 256-unit dense layer in the final configuration. The notebook does not provide enough measured results to claim a complete comparison against every alternative learning rate, optimizer, batch size, or dense-layer size listed in the manual, so no unsupported numerical comparison is reported.

17. Mandatory Plots

The laboratory manual requires the following plots.

1. Sample CIFAR-10 images
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

All required plots were generated from the notebook and inserted into Tasks 1, 4 and 5 above. The notebook also produced an additional architecture-comparison plot, included below as a supplementary result.

18. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy (final epoch)	99.00%
Testing Accuracy	90.55%
Precision	0.9055
Recall	0.9055
F1-score	0.9053
Training Time	1084.36 s
Total Parameters	24,114,826

The final training accuracy of 99.00% and test accuracy of 90.55% show that the model learned strong discriminative representations. There is a 8.45 percentage-point gap between final training and testing accuracy, indicating some degree of overfitting, although the validation accuracy continued to improve during fine tuning.

18.2 Comparison of CNN Architectures

The notebook included the following comparison table, combining reference values for earlier architectures with the measured result for the ResNet50 experiment.

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	60K	~99 (MNIST)	Seconds
AlexNet	61M	~57 (CIFAR-10, scratch)	Minutes
VGG16	138M	~93 (CIFAR-10, TL)	~Hours
GoogleNet	6.8M	~94 (CIFAR-10, TL)	~Hours
ResNet50 (this run)	24,114,826	90.55	1084.36 s

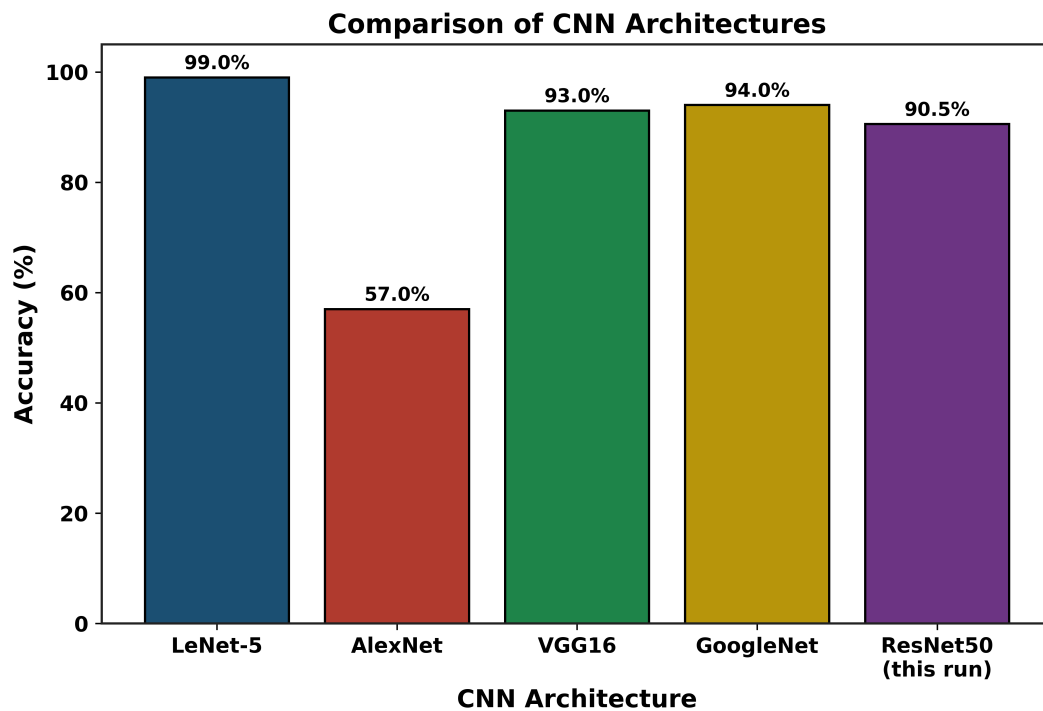


Figure 10: Comparison of CNN architecture accuracy using the values recorded in the notebook.

Inference: The comparison illustrates the increasing capacity and different design strategies of CNN architectures. However, the accuracy values are not a strictly controlled head-to-head benchmark: the notebook explicitly combines results from different datasets, training settings and literature references. The 90.55% ResNet50 value is the measured result of the present experiment.

19. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**
AlexNet demonstrated that a deep CNN trained with GPUs, ReLU activations, Dropout and data augmentation could achieve a major improvement on the ImageNet classification task. It helped establish deep learning as a practical approach for large-scale computer vision.
2. **Why does VGG16 use only 3×3 convolution filters?**
Stacking multiple 3×3 convolutions provides a larger effective receptive field while introducing more nonlinear activation stages. It also keeps the architecture simple and uses fewer parameters than using equivalently sized large filters in a single layer.
3. **Explain the advantages of the Inception module.**
The Inception module processes the same input using multiple receptive field sizes in parallel and concatenates their outputs. This enables multi-scale feature extraction while 1×1 convolutions reduce computational cost and parameter count.
4. **What is the purpose of residual learning?**
Residual learning makes very deep networks easier to optimize by learning a residual mapping $F(x)$ and adding the identity mapping x . The skip connection provides a direct path for information and gradients.
5. **Differentiate LeNet and ResNet.**
LeNet-5 is a small early CNN designed for handwritten digit recognition, with only a few convolution and pooling layers. ResNet uses many residual blocks and skip connections, allowing substantially deeper networks to be trained effectively.
6. **What is Transfer Learning?**
Transfer learning reuses features learned by a model on a large source dataset such as ImageNet and adapts them to a new target task. In this experiment, ImageNet-pretrained ResNet50 features were reused for CIFAR-10 classification.

7. Why is fine tuning required?

Fine tuning allows selected pretrained layers to adapt their learned features to the target dataset. In this experiment, unfreezing the final ResNet50 residual block improved validation accuracy from 88.86% to 91.38%.

8. Explain the difference between dilated convolution and transpose convolution.

Dilated convolution expands the receptive field by inserting gaps between kernel elements without directly increasing the kernel's number of parameters. Transpose convolution is a learnable upsampling operation used to increase the spatial dimensions of feature maps.

9. Why do pretrained models converge faster?

A pretrained model already contains useful low- and mid-level visual features. Therefore, the new classifier does not need to learn all visual representations from random initialization, reducing the amount of optimization required.

10. Compare the computational complexity of LeNet and ResNet.

LeNet has a small number of layers and parameters and therefore requires much less computation. ResNet50 is substantially deeper and has millions of parameters, but residual connections make the deeper architecture trainable and effective for complex image-recognition tasks.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

The present notebook completed the ResNet50 transfer-learning route, including frozen-base training, fine tuning, evaluation metrics, confusion matrix and architecture comparison. The other listed exercises were not recorded as completed runs in the notebook and therefore are not assigned unsupported numerical results here.

21. Source Code

The complete source code for this experiment is available at:

https://github.com/miruuuuuuuu/Deep-Learning-Lab/tree/main/Experiment4_CNNArch_039

22. Conclusion

This experiment studied the evolution of CNN architectures from LeNet-5 through AlexNet, VGG16, GoogleNet and ResNet, and demonstrated transfer learning using an ImageNet-pretrained ResNet50 on CIFAR-10. The final model used a frozen pretrained feature extractor followed by a custom classification head and was subsequently fine tuned by unfreezing the last residual block.

The frozen-base phase achieved 88.86% validation accuracy, while fine tuning improved validation accuracy to 91.38%. On the held-out test set, the final model achieved 90.55% accuracy, with weighted precision of 0.9055, weighted recall of 0.9055 and weighted F1-score of 0.9053. The results demonstrate that pretrained representations can provide strong performance while reducing the amount of training required from scratch. The confusion matrix and misclassified examples also show that visually similar CIFAR-10 classes remain challenging, leaving scope for further improvements through augmentation, architecture changes and systematic hyperparameter tuning.

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>